

NEIGE & SOLEIL

Application de gestion locative en station — Client lourd Java

Documentation technique

Architecture, modèle de données et déploiement

Champ	Valeur
Projet	Neige & Soleil (client lourd)
Document	Documentation technique
Version	1.0
Date	8 juin 2026
Statut	Interne

Sommaire

Sommaire	2
1. Présentation technique	3
2. Architecture applicative	3
2.1 Organisation en packages	3
2.2 Principe MVC	3
2.3 Classes principales	3
3. Modèle de données	4
3.1 Tables	4
3.2 Relations principales	4
3.3 Déclencheurs (triggers)	4
4. Accès aux données	5
4.1 Connexion JDBC	5
4.2 Exécution des requêtes	5
5. Flux fonctionnels clés	5
5.1 Authentification	5
5.2 Réservation	5
5.3 Facturation	5
6. Installation et déploiement	5
6.1 Prérequis	5
6.2 Mise en place de la base	5
6.3 Configuration de la connexion	5
7. Dette technique et améliorations	6

1. Présentation technique

Neige & Soleil est une application de bureau (client lourd) développée en Java avec une interface graphique Swing, connectée à une base de données MySQL. Elle suit une architecture en couches de type MVC (Modèle – Vue – Contrôleur).

Élément	Choix technique
Langage	Java
Interface graphique	Swing (JFrame, JPanel, JButton, JTable...)
Architecture	MVC — packages vue, controleur, modele
Base de données	MySQL (base neige_soleil)
Accès aux données	JDBC — pilote com.mysql.cj.jdbc.Driver
Type d'application	Client lourd installé sur poste

2. Architecture applicative

2.1 Organisation en packages

- **vue** : fenêtres et panneaux de l'interface (connexion, fenêtre générale, panneaux par module).
- **contrôleur** : classes métier (entités) et classe Contrôleur faisant le lien entre la vue et le modèle.
- **modele** : classe Modele (requêtes SQL) et classe BDD (gestion de la connexion JDBC).

2.2 Principe MVC

1. La Vue capte les actions de l'utilisateur (clics, saisies) et affiche les résultats.
2. Elle appelle la classe **Contrôleur**, qui sert de façade et délègue au **Modèle**.
3. Le **Modèle** construit et exécute les requêtes SQL via la classe **BDD**, puis renvoie des objets métier (Client, Appartement, Reservation...).

2.3 Classes principales

Classe	Package	Rôle
Neige_soleil	controleur	Point d'entrée (main) ; ouvre la connexion et la fenêtre générale.
Contrôleur	controleur	Façade exposant les opérations CRUD vers le modèle.
User, Client, Proprietaire, Personnel	controleur	Entités liées aux personnes.
Appartement, Materiel, Reservation	controleur	Entités du domaine locatif.
Modele	modele	Méthodes statiques de lecture/écriture en base.

Classe	Package	Rôle
BDD	modele	Chargement du pilote, connexion et déconnexion JDBC.
VueConnexion	vue	Écran d'authentification.
VueGenerale	vue	Fenêtre principale avec menu et zone de contenu.
Panel... (Client, Appartements, Reservation, Facture...)	vue	Panneau dédié à chaque module.

3. Modèle de données

3.1 Tables

Table	Contenu
User	Compte d'accès (nom, prénom, e-mail, mot de passe, rôle, téléphone).
PROPRIETAIRE	Propriétaires d'appartements (coordonnées, IBAN).
APPARTEMENT	Biens loués (type, surface, capacité, exposition, prix hebdo, propriétaire).
CONTRAT	Contrat associé à un appartement (dates, tarif saison, archivage).
EQUIPEMENT	Référentiel d'équipements d'appartement.
CLIENT	Clients locataires.
PERSONNEL	Membres du personnel.
RESERVATION	Séjours (dates, nb personnes, client, appartement, employé).
PAIEMENT	Paieement lié à une réservation (montant, date, mode).
MATERIEL	Matériel de ski (libellé, type, état, prix/jour).
DISPOSER	Association appartement ↔ équipement (n..n).
LOUER_MATERIEL	Location de matériel par un client (dates).

3.2 Relations principales

- `APPARTEMENT.id_proprio` → `PROPRIETAIRE` (un propriétaire possède plusieurs appartements).
- `CONTRAT.id_appart` → `APPARTEMENT` (relation 1'1).
- `RESERVATION` → `CLIENT`, `APPARTEMENT`, `PERSONNEL`.
- `PAIEMENT.id_reser` → `RESERVATION` (1'1, suppression en cascade).
- `DISPOSER` : table d'association entre `APPARTEMENT` et `EQUIPEMENT`.
- `LOUER_MATERIEL` → `CLIENT` et `MATERIEL`.

3.3 Déclencheurs (triggers)

La table `User` centralise les personnes. Des déclencheurs maintiennent la cohérence : toute insertion, mise à jour ou suppression dans `CLIENT` ou `PROPRIETAIRE` est répercutée dans `User` (avec le rôle correspondant). L'identifiant est calculé à partir du maximum existant.

4. Accès aux données

4.1 Connexion JDBC

La classe `BDD` charge le pilote MySQL puis ouvre une connexion à l'URL `jdbc:mysql://localhost/neige_soleil`. Les méthodes `seConnecter()` et `SeDeConnecter()` encadrent chaque opération.

4.2 Exécution des requêtes

La classe `Modele` expose des méthodes statiques (par exemple `selectWhereUser`, `insertReservation`, `recupererDonneesFacture`). Chaque méthode se connecte, exécute la requête via un `Statement`, lit le `ResultSet`, instancie des objets métier, puis se déconnecte.

5. Flux fonctionnels clés

5.1 Authentification

4. L'utilisateur saisit e-mail + mot de passe dans `VueConnexion`.
5. `Controleur.selectWhereUser` interroge la table `user`.
6. Si un utilisateur correspond, `VueGenerale` s'ouvre ; sinon un message d'erreur est affiché.

5.2 Réservation

7. Saisie des dates, du nombre de personnes, du client, de l'appartement et de l'employé.
8. `Controleur.insertReservation` insère l'enregistrement dans `RESERVATION`.

5.3 Facturation

9. `recupererDonneesFacture(id_reser)` récupère client, type d'appartement, prix et durée par une jointure `RESERVATION × CLIENT × APPARTEMENT`.
10. Calcul : `totalHT = prix × duree`, `tva = totalHT × 0,20`, `totalTTC = totalHT + tva`.

6. Installation et déploiement

6.1 Prérequis

- Un environnement d'exécution Java sur chaque poste.
- Un serveur MySQL accessible (local ou réseau).
- Le pilote JDBC MySQL dans le classpath de l'application.

6.2 Mise en place de la base

11. Exécuter le script de création de la base (tables + déclencheurs).
12. Créer au moins un compte utilisateur pour permettre la première connexion.

6.3 Configuration de la connexion

Les paramètres (serveur, base, utilisateur, mot de passe) sont actuellement définis dans la classe `Modele`. Il est recommandé de les externaliser dans un fichier de configuration et d'utiliser un compte applicatif dédié (voir documentation des incidents, INC-003).

7. Dette technique et améliorations

Les points suivants sont à traiter en priorité pour fiabiliser et sécuriser l'application (détaillés dans la documentation des incidents) :

- Passer aux requêtes préparées pour éliminer le risque d'injection SQL.
- Hacher les mots de passe au lieu de les stocker en clair.
- Externaliser et restreindre les identifiants de connexion à la base.
- Corriger les accesseurs `getId_perso()` et `getRole()` de la classe `User`.
- Conditionner l'affichage des modules selon le rôle de l'utilisateur.
- Homogénéiser le calcul de facturation (unité de prix vs durée).
- Charger les images comme ressources embarquées et mutualiser la connexion à la base.